

✓ Canadian Food Tax Optimization for CO₂ Reduction

APM348 Winter 2026

This notebook runs the full optimization pipeline. Upload the `food_tax_optimization.zip` file when prompted in the first cell.

```
# =====
# CELL 1: Upload and extract the project zip
# =====
from google.colab import files
import zipfile, os, shutil

# Clean up any previous runs
if os.path.exists('food_tax_optimization'):
    shutil.rmtree('food_tax_optimization')

print('Please upload food_tax_optimization.zip:')
uploaded = files.upload()

# Extract
zip_name = list(uploaded.keys())[0]
with zipfile.ZipFile(zip_name, 'r') as z:
    z.extractall('.')

# Verify
project_dir = 'food_tax_optimization'
print(f'\nExtracted files:')
for f in sorted(os.listdir(project_dir)):
    print(f' {f}')
print('\nSetup complete!')
```

Please upload food_tax_optimization.zip:

[Choose Files](#) food_tax_optimization (1).zip

food_tax_optimization (1).zip(application/x-zip-compressed) - 494696 bytes, last modified: 3/22/2026 - 100% done

Saving food_tax_optimization (1).zip to food_tax_optimization (1) (1).zip

Extracted files:

```
README.md
categories.py
config.py
constraints.py
data
data_loader.py
demand_model.py
emissions.py
generate_baseline_template.py
main.py
optimizer.py
output
results.py
visualize.py
```

Setup complete!

```
# =====
# CELL 2: Add project to Python path and import all modules
# =====
import sys
sys.path.insert(0, 'food_tax_optimization')

import numpy as np
import matplotlib.pyplot as plt

from categories import CATEGORIES, N_CATEGORIES
from config import OptimizationConfig
from data_loader import load_elasticity_matrix, load_emissions, load_baseline_data
from demand_model import LAIDSMoel
from emissions import total_emissions, baseline_emissions, emissions_reduction
from constraints import evaluate_constraints
from optimizer import optimize_tax, sensitivity_analysis
from results import format_results, results_to_dataframe
from visualize import (
    plot_optimal_taxes, plot_emissions_comparison,
    plot_quantity_changes, plot_sensitivity, plot_tax_heatmap
```

```
)

print(f'All modules imported. {N_CATEGORIES} food categories loaded.')
```

```
All modules imported. 14 food categories loaded.
```

```
# =====
# CELL 3: Load all data
# =====
data_dir = 'food_tax_optimization/data'

E = load_elasticity_matrix(f'{data_dir}/elasticities.xlsx')
e = load_emissions(f'{data_dir}/emissions.csv')
baseline = load_baseline_data(f'{data_dir}/baseline.csv')

print(f'Elasticity matrix: {E.shape}')
print(f'Emissions vector: {e.shape}')
print(f'Budget shares sum: {baseline["budget_shares"].sum():.6f}')
print(f'Total expenditure: ${baseline["total_expenditure"]:.2f}')
print()
print('Per-category emissions (kg CO2-eq/unit):')
for i, cat in enumerate(CATEGORIES):
    print(f' {cat:<28} {e[i]:>8.3f}')
```

```
Elasticity matrix: (14, 14)
Emissions vector: (14,)
Budget shares sum: 0.999998
Total expenditure: $5708.01
```

```
Per-category emissions (kg CO2-eq/unit):
Beef                99.477
Poultry              9.866
Pork                12.306
Other meat          39.722
Fish                20.249
Dairy               20.110
Eggs                 4.669
Bakery               1.574
Cereal               2.453
Fruit                0.853
Vegetable            0.851
Fats and oils        5.287
Non-alcoholic beverages 14.752
Other food           7.757
```

```
# =====
# CELL 4: Build LA-AIDS demand model
# =====
model = LAIDSMoel(
    elasticity_matrix=E,
    baseline_budget_shares=baseline['budget_shares'],
    baseline_prices=baseline['prices'],
    total_expenditure=baseline['total_expenditure'],
)

# Sanity check: zero tax should reproduce baseline
result0 = model.predict_with_tax(np.zeros(N_CATEGORIES))
print(f'Zero-tax expenditure: ${result0["expenditure"]:.2f}')
print(f'Baseline expenditure: ${baseline["total_expenditure"]:.2f}')
print(f'Match: {abs(result0["expenditure"] - baseline["total_expenditure"]) < 0.1}')
print()
print('Expenditure elasticities (from Cournot aggregation):')
for i, cat in enumerate(CATEGORIES):
    print(f' {cat:<28} {model.expenditure_elasticity[i]:>8.4f}')
```

```
Zero-tax expenditure: $5708.00
Baseline expenditure: $5708.01
Match: True
```

```
Expenditure elasticities (from Cournot aggregation):
Beef                0.8385
Poultry              0.8420
Pork                0.8012
Other meat          0.9027
Fish                0.7940
Dairy               1.0826
Eggs                 0.6113
Bakery               0.9410
Cereal               0.8945
```

Fruit	1.2840
Vegetable	1.3080
Fats and oils	0.7464
Non-alcoholic beverages	0.8969
Other food	1.0577

```
# =====
# CELL 5: Run optimization
# =====
# You can modify these constraint thresholds:
config = OptimizationConfig(
    max_tax_per_product=0.20,      # 20% max tax on any single product
    min_tax_per_product=0.0,       # no subsidies (set negative to allow)
    max_total_tax=0.10,           # 10% max weighted-average tax
    max_budget_impact=0.05,        # 5% max household budget increase
    min_gdp_fraction=0.95,        # food GDP cannot drop below 95% of baseline
    optimizer_tol=1e-12,
    optimizer_maxiter=5000,
)

base_e = baseline_emissions(model, e)
print(f'Baseline total CO2 emissions: {base_e:.2f} kg CO2-eq')
print()

opt = optimize_tax(model, e, config)
print(f'Optimizer converged: {opt.success}')
print(f'Optimizer message: {opt.message}')
print(f'Optimized emissions: {opt.fun:.2f} kg CO2-eq')
print(f'Reduction: {(base_e - opt.fun)/base_e*100:.2f}%')
```

Baseline total CO2 emissions: 11582.29 kg CO2-eq

Optimizer converged: True
 Optimizer message: Optimization terminated successfully
 Optimized emissions: 10046.96 kg CO2-eq
 Reduction: 13.26%

```
# =====
# CELL 6: Full results report
# =====
report = format_results(opt, model, e, config)
print(report)
```

```
=====
Optimizer converged: True
Optimizer message: Optimization terminated successfully
```

```
--- Emissions Summary ---
Baseline total CO2: 11582.2911
Optimized total CO2: 10046.9590
Absolute reduction: 1535.3321
Percent reduction: 13.26%
```

```
--- Optimal Tax Rates ---
Category                Tax Rate    Tax %
-----
Beef                    0.20000    20.00%
Poultry                 0.00000     0.00%
Pork                    0.00000     0.00%
Other meat              0.20000    20.00%
Fish                    0.20000    20.00%
Dairy                   0.20000    20.00%
Eggs                    0.20000    20.00%
Bakery                  0.00550     0.55%
Cereal                  0.00000     0.00%
Fruit                   0.00000     0.00%
Vegetable               0.00000     0.00%
Fats and oils           0.04567     4.57%
Non-alcoholic beverages 0.20000    20.00%
Other food              0.18577    18.58%
```

```
--- Economic Impact ---
Baseline food expenditure: 5708.01
New food expenditure:      5679.41
Budget impact:             -0.50%

Food-sector GDP change:    -0.50%
```

Beef	23.93	21.20	-11.41%	21.0%
Poultry	32.59	32.92	1.02%	3.2%
Pork	23.60	23.48	-0.52%	2.9%
Other meat	11.00	9.33	-15.14%	3.7%
Fish	12.44	11.71	-5.81%	2.4%
Dairy	177.78	149.90	-15.68%	30.0%
Eggs	28.29	26.56	-6.11%	1.2%
Bakery	145.25	138.30	-4.78%	2.2%
Cereal	79.78	78.49	-1.61%	1.9%
Fruit	187.43	179.64	-4.15%	1.5%
Vegetable	205.33	196.73	-4.19%	1.7%
Fats and oils	19.09	18.16	-4.90%	1.0%
Non-alcoholic beverages	147.20	121.12	-17.72%	17.8%
Other food	149.83	124.29	-17.05%	9.6%

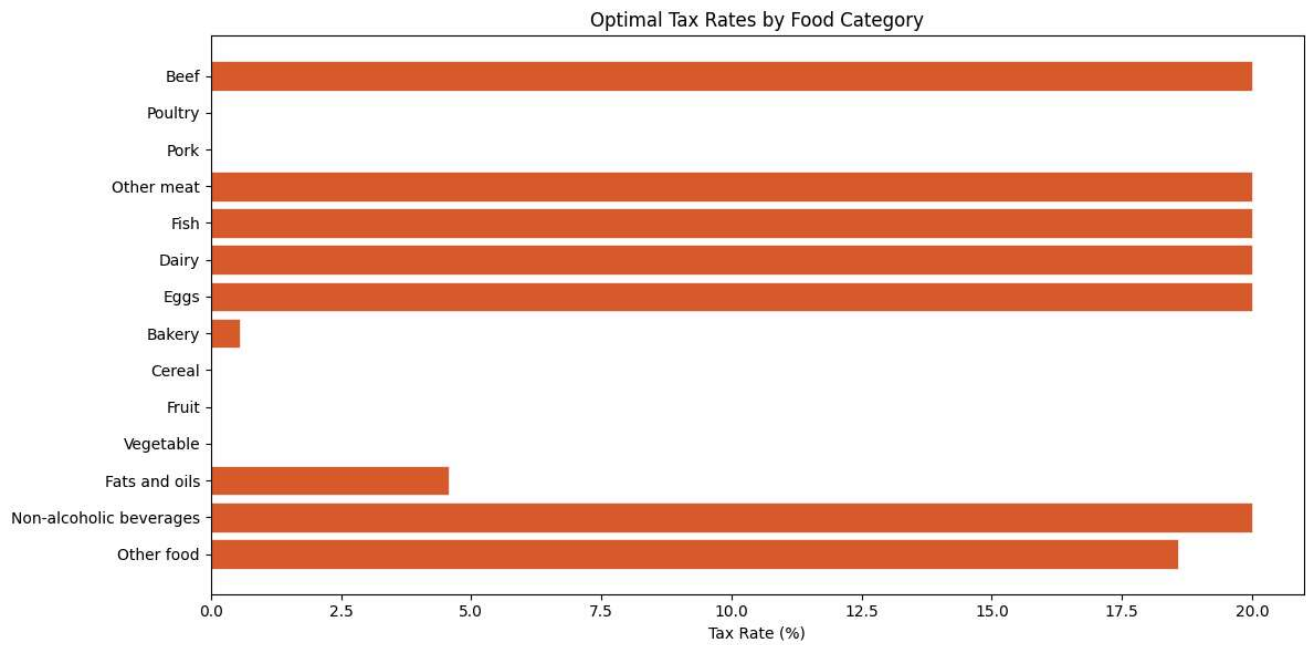
--- Constraint Status ---

max_total_weighted_tax	slack=0.000000 [OK]
max_budget_impact	slack=0.055008 [OK]
min_gdp_fraction	slack=256.816057 [OK]

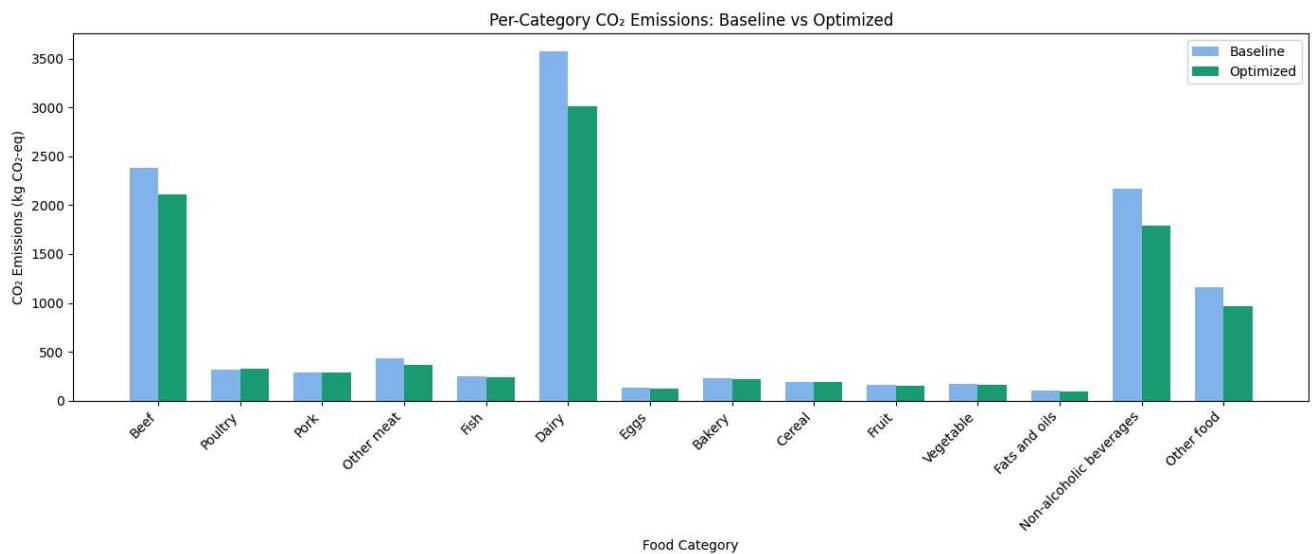
```
# =====
# CELL 7: Results as DataFrame
# =====
df = results_to_dataframe(opt, model, e)
df.style.format({
    'tax_rate': '{:.5f}',
    'tax_percent': '{:.2f}%',
    'baseline_price': '${:.2f}',
    'new_price': '${:.2f}',
    'baseline_quantity': '{:.1f}',
    'new_quantity': '{:.1f}',
    'quantity_change_pct': '{:.2f}%',
    'baseline_co2': '{:.1f}',
    'new_co2': '{:.1f}',
    'co2_reduction': '{:.1f}',
})
```

	category	tax_rate	tax_percent	baseline_price	new_price	baseline_quantity	new_quantity	quantity_change_pct	baseline_c
0	Beef	0.20000	20.00%	\$15.00	\$18.00	23.9	21.2	-11.41%	2380
1	Poultry	0.00000	0.00%	\$8.50	\$8.50	32.6	32.9	1.02%	327
2	Pork	0.00000	0.00%	\$10.00	\$10.00	23.6	23.5	-0.52%	290
3	Other meat	0.20000	20.00%	\$14.00	\$16.80	11.0	9.3	-15.14%	437
4	Fish	0.20000	20.00%	\$16.00	\$19.20	12.4	11.7	-5.81%	257
5	Dairy	0.20000	20.00%	\$4.50	\$5.40	177.8	149.9	-15.68%	3575
6	Eggs	0.20000	20.00%	\$3.50	\$4.20	28.3	26.6	-6.11%	132
7	Bakery	0.00550	0.55%	\$4.00	\$4.02	145.3	138.3	-4.78%	228
8	Cereal	0.00000	0.00%	\$4.50	\$4.50	79.8	78.5	-1.61%	195
9	Fruit	0.00000	0.00%	\$3.50	\$3.50	187.4	179.6	-4.15%	159
10	Vegetable	0.00000	0.00%	\$3.00	\$3.00	205.3	196.7	-4.19%	174
11	Fats and oils	0.04567	4.57%	\$5.50	\$5.75	19.1	18.2	-4.90%	100
	Non-								

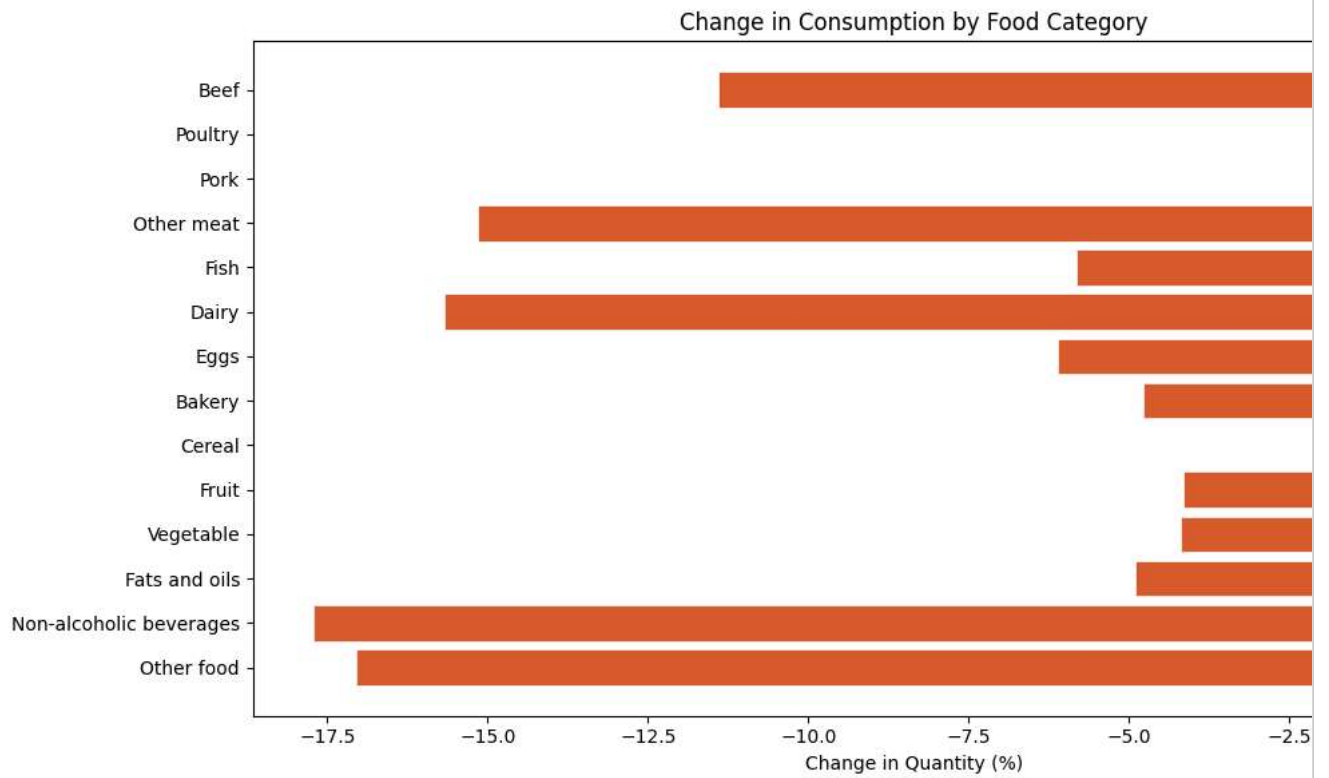
```
# =====
# CELL 8: Visualization - Optimal Tax Rates
# =====
fig = plot_optimal_taxes(opt)
plt.show()
```



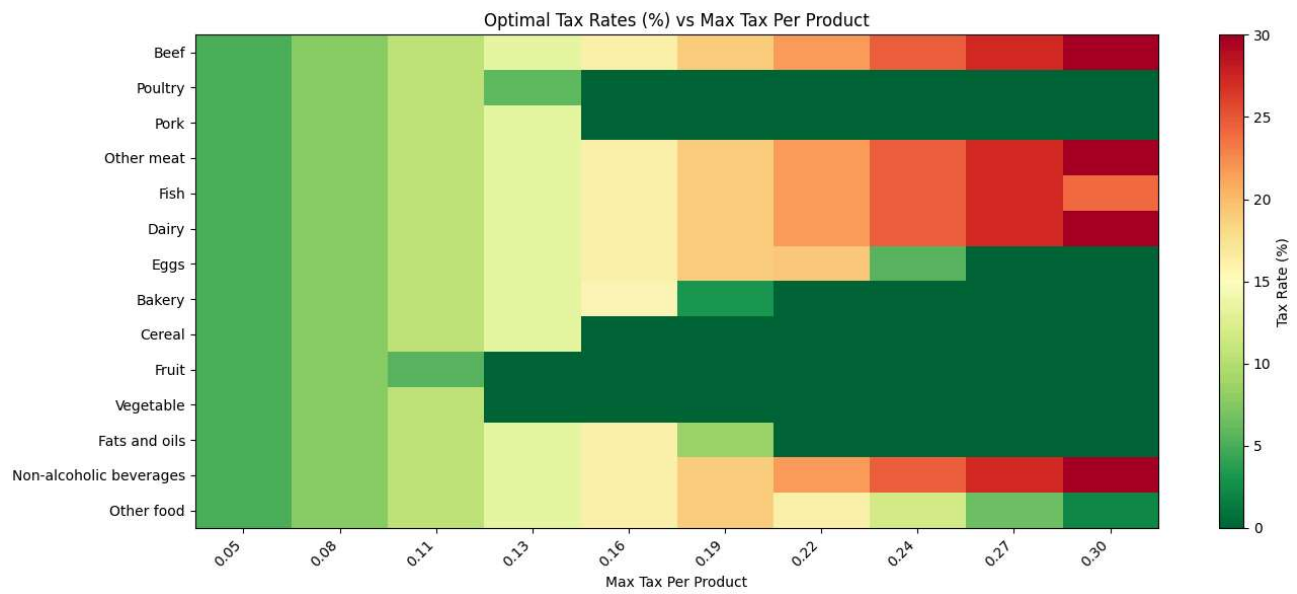
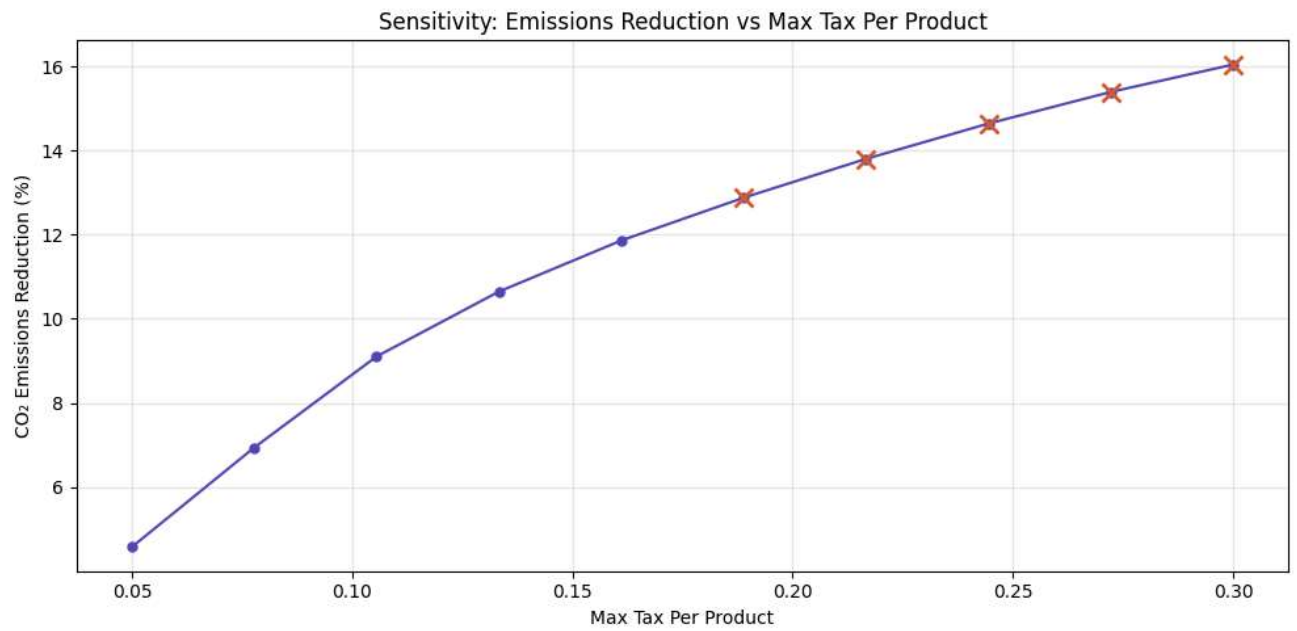
```
# =====
# CELL 9: Visualization - Emissions Comparison
# =====
fig = plot_emissions_comparison(opt, model, e)
plt.show()
```



```
# =====
# CELL 10: Visualization - Quantity Changes
# =====
fig = plot_quantity_changes(opt, model)
plt.show()
```



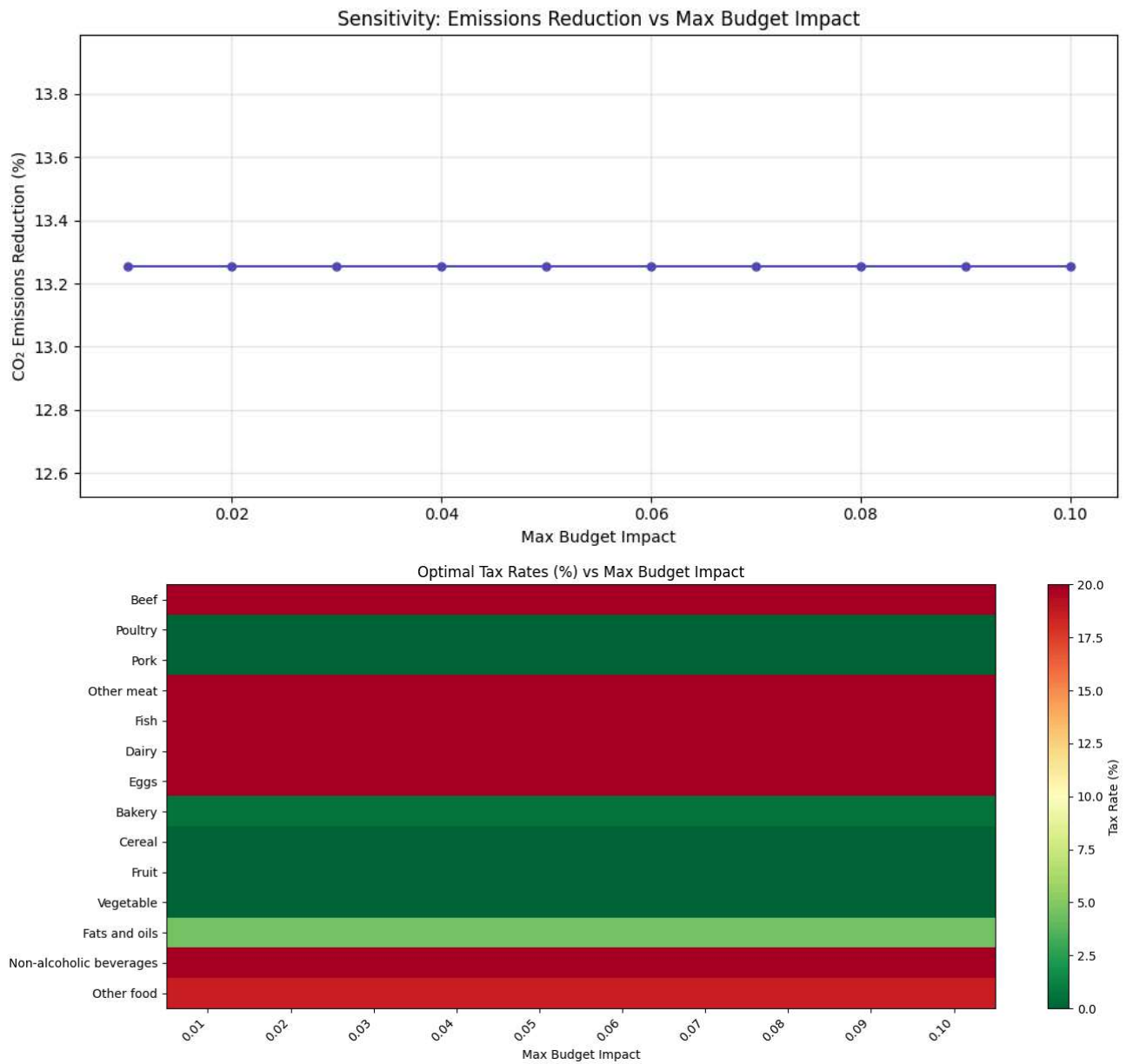
```
# =====  
# CELL 11: Sensitivity Analysis - Max Tax Per Product  
# =====  
tax_sweep = np.linspace(0.05, 0.30, 10)  
sa_tax = sensitivity_analysis(model, e, config, 'max_tax_per_product', tax_sweep)  
  
fig = plot_sensitivity(sa_tax, 'max_tax_per_product')  
plt.show()  
  
fig = plot_tax_heatmap(sa_tax, 'max_tax_per_product')  
plt.show()
```



```
# =====
# CELL 12: Sensitivity Analysis - Max Budget Impact
# =====
budget_sweep = np.linspace(0.01, 0.10, 10)
sa_budget = sensitivity_analysis(model, e, config, 'max_budget_impact', budget_sweep)

fig = plot_sensitivity(sa_budget, 'max_budget_impact')
plt.show()

fig = plot_tax_heatmap(sa_budget, 'max_budget_impact')
plt.show()
```



```
# =====
# CELL 13: Sensitivity Analysis - Max Total Tax
# =====
total_tax_sweep = np.linspace(0.02, 0.20, 10)
sa_total = sensitivity_analysis(model, e, config, 'max_total_tax', total_tax_sweep)

fig = plot_sensitivity(sa_total, 'max_total_tax')
plt.show()

fig = plot_tax_heatmap(sa_total, 'max_total_tax')
plt.show()
```